

Line Following & Obstacle Avoidance Control

01



Engineering Aspect

02



Hardware

03



Code

04



Evaluation

Team **A3**

Team Members: **Md Naimur Rahman & Adham Elsaygh & Hany Chowdhury & Torikul islam**

Presenters: **Md Naimur Rahman & Adham Elsaygh**

Systems Engineering Aspect

01 

System Integration

- **Mechanics:** Chassis, motor drive systems, and wheels.
- **Electronics:** Microcontroller, IR sensors, and ultrasonic sensors.
- **Software:** Threshold-based navigation and obstacle avoidance logic.

02 

What vs. How

- **Milestone 1:** Continuous track line-following.
- **Milestone 2:** Object detection and avoidance.

03 

SysML Verification

**Standardized visual modeling
prevents integration mismatches**

Hardware Setup & Integration

01 (••)

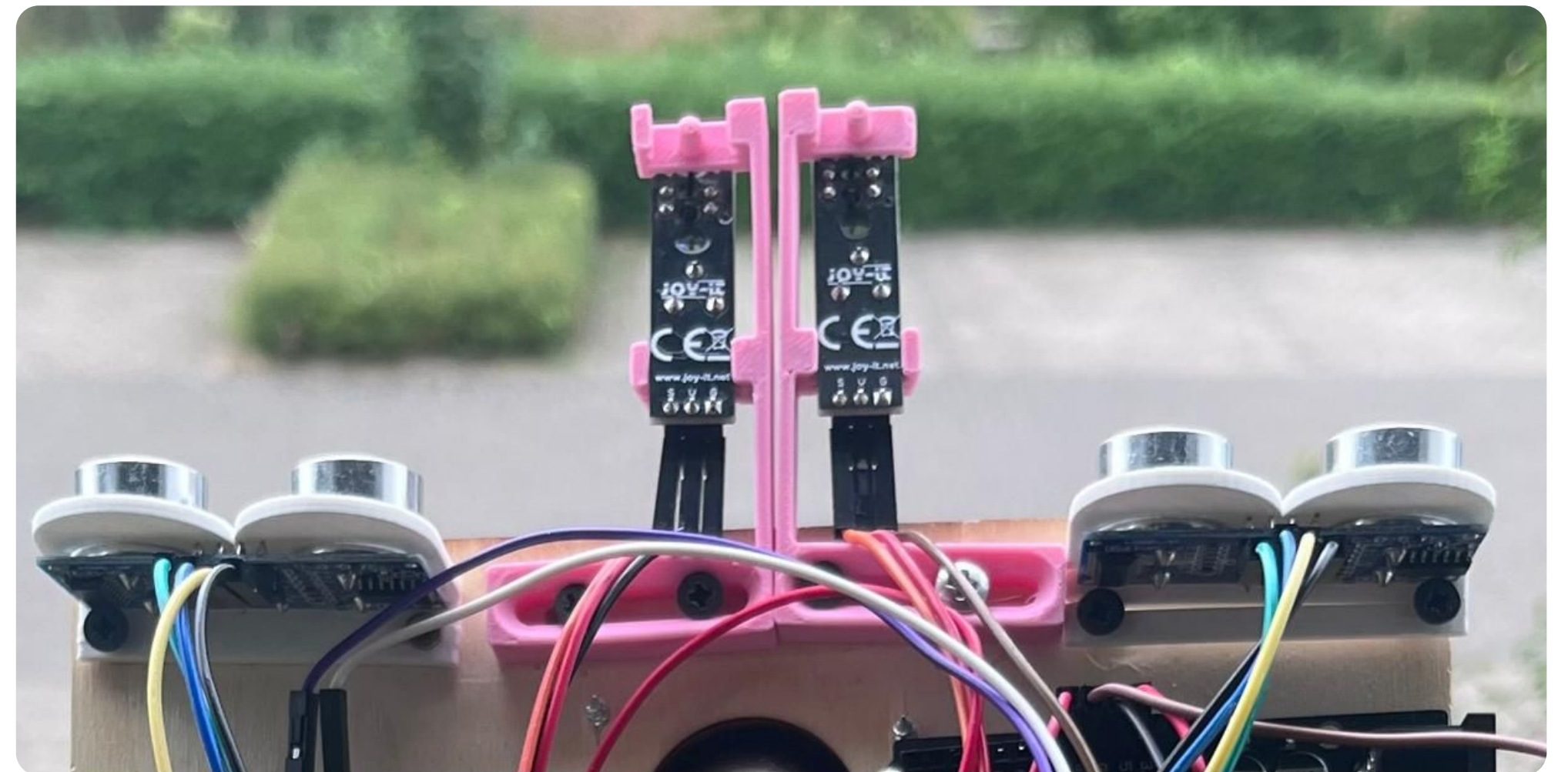
Line Following (IR)

Dual infrared sensor module configured to detect reflectivity differences for real-time track following and steering adjustments

02 ©

Obstacle Detection (Ultrasonic)

Ultrasonic transceiver calculating real-time distance to forward obstructions to initiate path planning.



03 

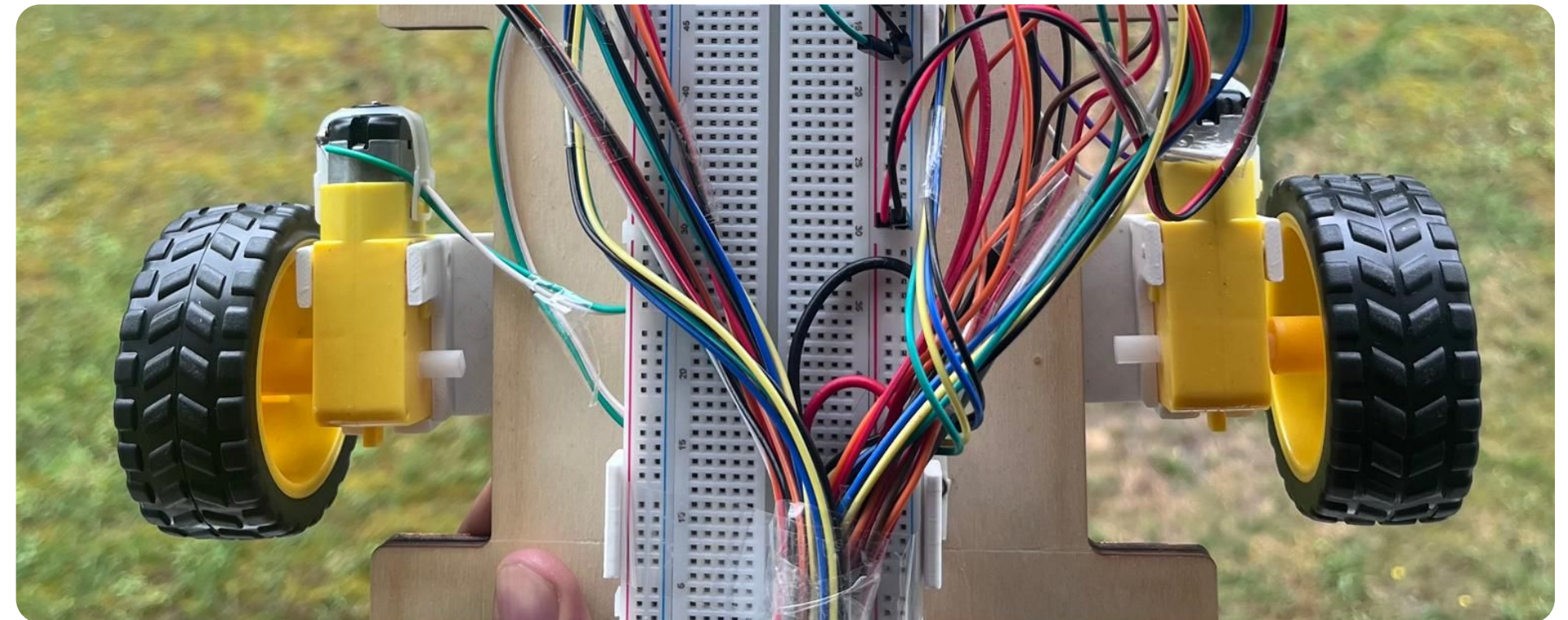
Motor Setup

High-torque DC motors configured for precise speed and directional mobility.

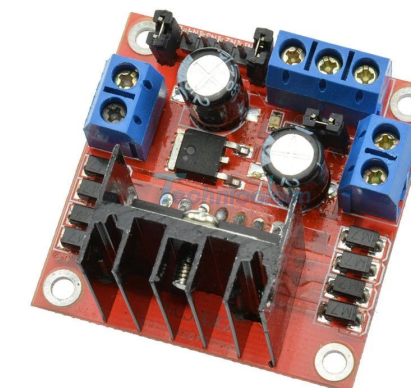
04 

Motor Driver (L298N)

Dual-channel H-bridge controller managing voltage and direction for the drive system.



Technoclaim



05

Voltage Regulation Flow



01. Power Source

LiPo Battery (11.1V)

High-capacity power source providing raw DC voltage to initiate the system's power chain.



02. Power Control

Toggle Switch

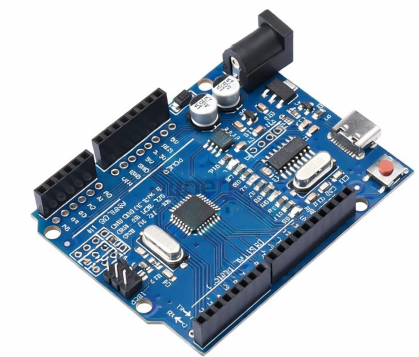
Acts as a physical switch to safely control and interrupt the main power supply line.



03. Regulation

LM2596 Buck Converter

Efficiently steps down the variable battery voltage to a safe, constant 5V output flow.



04. Output Load

Arduino Uno

Receives regulated 5V power to safely drive the logic, sensors, and microcontroller circuits.

Code Implementation

Pin & Speed Configurations

```
// IR Sensors
const int irLeftPin = 2;
const int irRightPin = 12;

// Ultrasonic Sensors (Your Custom Pins)
const int trigLeft = 13;
const int echoLeft = 4;
const int trigRight = 6;
const int echoRight = 7;

// Motor A (Left Motor Control)
const int ena = 3; // PWM pin for speed
const int in1 = 8; // Direction pin 1
const int in2 = 9; // Direction pin 2

// Motor B (Right Motor Control)
const int in3 = 10; // Direction pin 1
const int in4 = 11; // Direction pin 2
const int enb = 5; // PWM pin for speed

// --- Speed Adjustments ---
const int leftMotorSpeed = 70;
const int rightMotorSpeed = 79; // 80 + 17 to match the left motor's output
```

System Initialization

```
char lastState = 'F';

void setup() {
    Serial.begin(9600);

    // Initialize IR Sensors
    pinMode(irLeftPin, INPUT_PULLUP);
    pinMode(irRightPin, INPUT_PULLUP);

    // Initialize Ultrasonic Sensors
    pinMode(trigLeft, OUTPUT);
    pinMode(echoLeft, INPUT);
    pinMode(trigRight, OUTPUT);
    pinMode(echoRight, INPUT);

    // Initialize Motors
    pinMode(ena, OUTPUT);
    pinMode(enb, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
}
```

Line Following Logic

```
if (leftSensor == 1 && rightSensor == 1) {  
    moveForward();  
    lastState = 'F';  
}  
else if (leftSensor == 1 && rightSensor == 0) {  
    turnLeft();  
    lastState = 'L';  
}  
else if (leftSensor == 0 && rightSensor == 1) {  
    turnRight();  
    lastState = 'R';  
}  
else if (leftSensor == 0 && rightSensor == 0) {  
    // Completely lost the track! Use memory to find it.  
    if (lastState == 'L') {  
        spinLeft();  
    }  
    else if (lastState == 'R') {  
        spinRight();  
    }  
    else {  
        moveForward();  
    }  
}  
  
delay(10);
```


Obstacle Detection & Avoidance

```
if ((distanceLeft > 0 && distanceLeft <= obstacleDistance) ||
    (distanceRight > 0 && distanceRight <= obstacleDistance)) {

    motorStop();
    delay(50); // Brief pause to let motors settle (reduces electrical noise)

    // Read them one more time to verify
    distanceLeft = getDistance(trigLeft, echoLeft);
    delay(30);
    distanceRight = getDistance(trigRight, echoRight);

    // If it STILL sees the obstacle, execute the detour
    if ((distanceLeft > 0 && distanceLeft <= obstacleDistance) ||
        (distanceRight > 0 && distanceRight <= obstacleDistance)) {

        motorStop();
        delay(500);

        DriftLeft();
        delay(600);

        DriftRight();
        while (digitalRead(irLeftPin) == 0 && digitalRead(irRightPin) == 0) {
            // Let the motors run until a sensor finds the line
            delay(10);
        }

        // Once the loop breaks, at least one sensor found the line!
        return;
    }
}
```


Motor Control Values

```
void motorStop() {  
    analogWrite(ena, 0);  
    analogWrite(enb, 0);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
}  
  
void DriftLeft() {  
    analogWrite(ena, 55);  
    analogWrite(enb, 95);  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}  
  
void DriftRight() {  
    analogWrite(ena, 75);  
    analogWrite(enb, 50);  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}
```

```
void spinRight() {  
    analogWrite(ena, 85);  
    analogWrite(enb, 94);  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, HIGH);  
}  
  
void spinLeft() {  
    analogWrite(ena, 85);  
    analogWrite(enb, 94);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}
```

Testing and Evaluation

01. Motor Imbalance Issue

Problems noticed when testing:

During initial track runs, the prototype failed to maintain a straight trajectory despite receiving uniform control signals.

- The vehicle consistently drifted and veered off-course to one side.
- DC motors exhibited slight physical and electrical performance differences under identical voltage loads.
- Required a software-level speed calibration to equalize actual physical thrust.

02. Software Calibration

To resolve the drift, the motor speed constants were adjusted independently in the firmware setup to ensure balanced mechanical output:

```
// --- Speed Adjustments ---  
const int leftMotorSpeed = 70;  
const int rightMotorSpeed = 79; // Balanced to match left motor's output
```

- Left motor base speed set to **70** as reference.
- Right motor boosted to **79** to offset mechanical deficiency.

Thank You

Systems and Prototyping Project